Vita Care Hospital

Technical Design Document

Patient booking & AI assistant

Document ID	VCH-TDD-001
Version	1.0
Status	Draft — for stakeholder & technical review
Date	16 May 2026
Author	[Name / team]
Distribution	Internal review; GitHub org Vita-Care-Hospital

This document describes the design of a demonstration hospital digital front door: online appointment booking, a document-grounded assistant, and operator tooling. It is not a regulated clinical product.

Table of Contents

1. Executive summary

Vita Care Hospital is a working demonstration of how a mid-size hospital could present a modern patient-facing experience: browse departments, book an appointment, and ask questions through an assistant that respects hospital-specific documents. The build is intentionally small enough to run on a laptop yet complete enough to show integration patterns that matter in real programmes—separate services, explicit contracts, and guardrails around AI-generated claims.

Problem addressed

- Patients expect self-service booking and instant answers; hospitals need those channels without giving an AI direct access to clinical systems.
- Reviewers need evidence that booking outcomes and policy answers can be traced to structured APIs and ingested documents—not invented in the model.

Design intent

- One browser experience; three deployable backends so teams can evolve UI, booking, and AI independently.
- Bookings always flow through the same JSON API whether the user clicks cards or chats in natural language.
- Hospital FAQs are answered from uploaded documents; when nothing matches, the assistant refuses to fabricate policy.

Differentiators for this demo

- Tool-grounded booking: the assistant can only confirm an appointment when the booking API returns a real identifier.
- Operator-visible knowledge pipeline: upload, chunk, replace-by-filename, and see counts in admin.
- Dual booking paths: guided cards (no LLM cost) plus free-text chat for portfolio storytelling.

What this is not: production identity management, HIPAA-grade controls, or clinical triage. Those boundaries are stated explicitly in Section 5 so reviewers can separate demo merit from production gaps.

2. Business context & review purpose

The solution targets portfolio, hiring, and architecture conversations—not live patient traffic. Stakeholders should read Sections 1, 4, and 5 first; engineering detail and API listings sit in the appendices for those validating implementation depth.

Stakeholder outcomes this design supports

- Product / operations: see a credible booking funnel and admin view without a monolithic legacy rewrite.

- Architecture: see service boundaries, data ownership, and where AI sits relative to transactional APIs.
- Risk / compliance reviewers: see accepted limitations (synthetic data, regex emergency gate, optional admin token) before any pilot discussion.
- Engineering: follow diagrams and appendices to reproduce the stack locally or extend toward production.

Demo relevance: the same flows shown in the recorded walkthrough—home → book → assistant Q&A → admin ingest—map directly to Figures 1–5 and Appendix A screenshots.

3. Solution overview

At a glance, the platform delivers three patient-visible capabilities and one operator capability:

Capability	User value	How it is delivered
Department discovery	Patients see what the hospital offers before committing to a slot.	React home page backed by a departments API.
Appointment booking	Reduces front-desk load for routine scheduling.	Manual form or guided chat cards; both call the same booking API.
AI assistant	Answers hospital-specific and general health questions in one widget.	Chat service with retrieval over ingested docs and tools for live slot lookup.
Knowledge & bookings admin	Operators upload policies/FAQs and inspect demo bookings.	Hidden admin route; optional bearer token on ingest endpoints.

4. Architecture overview

Figure 1 is the single picture a reviewer should remember: the browser talks to one origin; a gateway forwards to a booking API and a chat API; vectors and SQLite hold knowledge and appointments; hosted or local models power embeddings and chat.

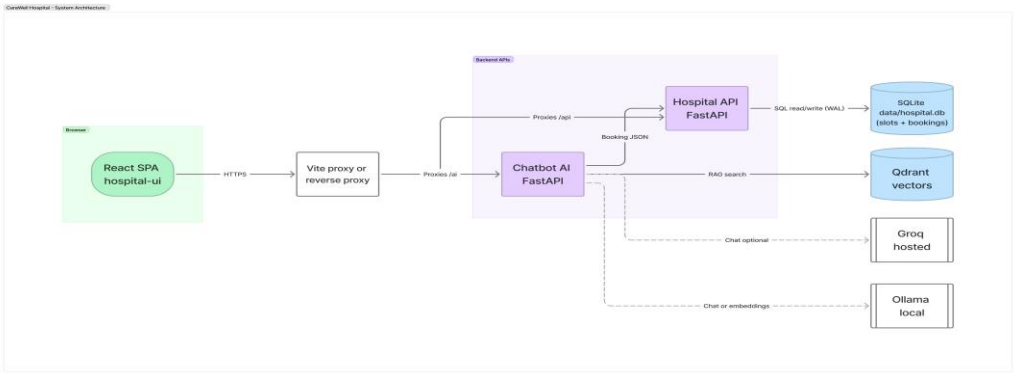


Figure 1 — End-to-end runtime (browser to data stores)

4.1 Logical components (summary)

Detailed ports and responsibilities are in **Appendix C — Component reference**.

Layer	Role
Experience	React SPA — booking UI, floating assistant, admin.
Transactional API	FastAPI service — departments, slots, create/cancel booking (SQLite).
Intelligence API	FastAPI service — chat, RAG ingest, tool dispatch to booking API.
Data & models	Qdrant (vectors), Ollama (embeddings), optional Groq (hosted chat).

5. Risks, limitations & production boundaries

The demo optimises for clarity and repeatability. The table below states what we accept for a portfolio build versus what would block a go-live without further investment.

Area	Demo position	Production expectation
Patient data	Synthetic; names are free text.	PHI handling, consent, retention policies.
Authentication	No patient login; admin optionally token-gated.	OIDC, RBAC, audit trails.
AI safety	Regex emergency shortcut; post-reply booking guards.	Clinical triage pathways; formal eval harness.
Availability	Local Docker / dev processes.	Managed DB, HA vector store, SLOs.
Conversation memory	History truncated for provider token limits.	Server-side booking state; summarised memory.
Knowledge freshness	Manual upload / script seed.	CMS integration, versioning, approval workflow.

6. Design decisions

Each row documents a deliberate choice for this demo: what we decided, why, what it buys stakeholders in review, and what we would not claim in production without further work.

Topic	Decision	Rationale	Benefit (demo)	Limitation / risk accepted
Service split	Three repositories: UI, booking API, chat API.	Mirrors how hospitals often separate digital channels from	Teams can demo or replace one layer without rewiring	More repos to clone; integration tested via HTTP

		clinical systems of record.	the whole stack.	only.
Booking persistence	SQLite file for slots and appointments.	Survives restarts; easy reset for repeated demos.	Credible “real” booking IDs in screenshots and chat.	Not suited to concurrent hospital-scale load; migrations are manual.
Agent orchestration	Hand-written tool loop in chat service; LangChain for ingest/RAG glue only.	Interviewers and auditors can read one module for control flow.	Faster to explain guardrails; easier to patch provider quirks.	More bespoke code than an off-the-shelf agent framework.
Model provider	Groq preferred for demos; Ollama for embeddings and offline fallback.	Low latency for live walkthroughs without GPU setup for chat.	Smooth presenter experience on typical laptops.	Hosted dependency, API keys, and provider rate limits.
Knowledge store	Qdrant + local embeddings (Ollama).	Avoids extra paid services for vector search in a portfolio setup.	Shows a complete RAG path operators can touch in admin.	Cold start and hardware sensitivity; not multi-region by default.
AI booking truth	Tools call booking API; guards block ungrounded confirmation text.	Reduces “hallucinated appointment” risk in demos.	Builds reviewer confidence that AI is bounded by APIs.	Does not stop a determined model from misleading on non-booking topics.
Admin security	Hidden /admin route; optional ADMIN_TOKEN on ingest APIs.	Enough to show ingest is not public-by-default in a serious design.	Quick operator demo without full IAM project.	UI hiding is not authorization; token is opt-in.

7. Capability flows (technical narrative)

The following sections support engineering walkthroughs. They assume familiarity with REST APIs; request/response detail is deferred to **Appendix B — HTTP interface reference**.

7.1 Knowledge ingestion

Operators upload PDFs or text from admin (or seed from a knowledge folder). Documents are chunked, embedded, and stored in Qdrant. Re-uploading the same filename replaces prior chunks so tuning chunk size does not leave orphan vectors.

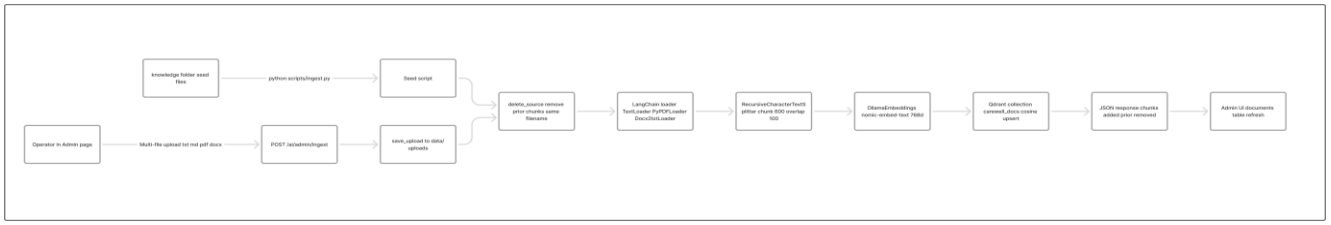


Figure 2 — Knowledge-base ingestion

```
PSEUDOCODE (sketch):
  removed ← delete_vectors_for_source(filename)
  chunks ← split(embed(load(upload)))
  upsert chunks → Qdrant
```

Full pseudocode and code snippets: Appendix E.1 Knowledge Ingest.

7.2 Booking via the assistant

The model never receives database credentials. It requests departments and slots through tools; only API responses count as truth. After the model replies, guards verify that any claimed booking ID appeared in a tool result and that mentioned times existed in the last slot list.

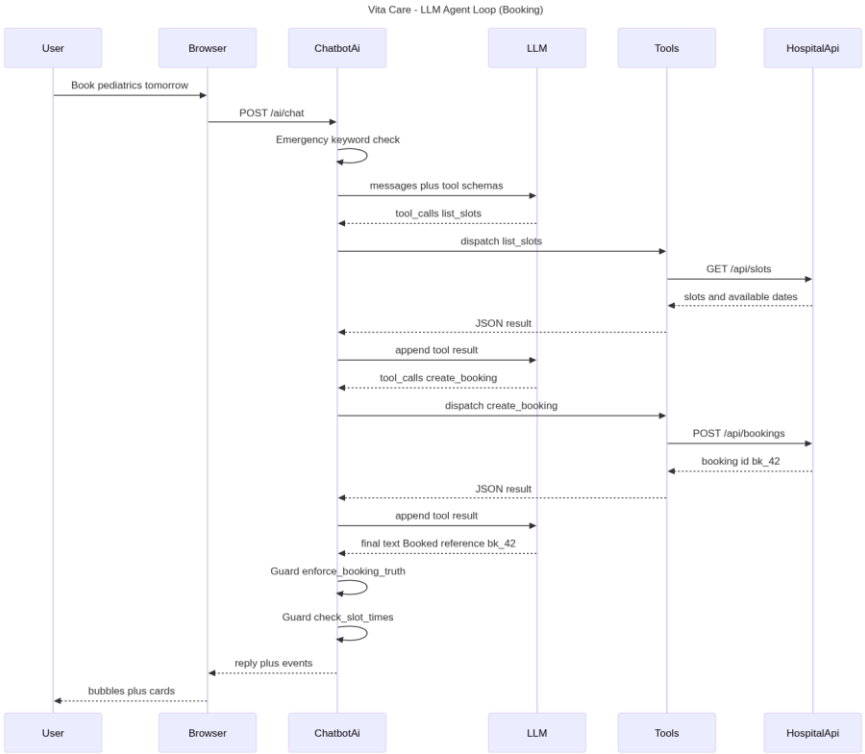


Figure 3 — AI-assisted booking (tool loop)

```
PSEUDOCODE (sketch):
  FOR each tool_call: result ← booking API (JSON)
  AFTER model reply:
    IF claims "confirmed" BUT no create_booking.id → replace reply
    IF mentions times ∉ last list_slots → append correction
```

Full pseudocode and code snippets: Appendix E.3 Booking via tools & post-reply guards *and* E.5 Booking transaction — SQLite.

7.3 Hospital FAQ (RAG)

Hospital-specific questions trigger document search first. Answers cite retrieved snippets; empty indexes produce a refusal rather than invented policy. General health questions receive cautious general information plus the standard not-a-doctor disclaimer in the UI.

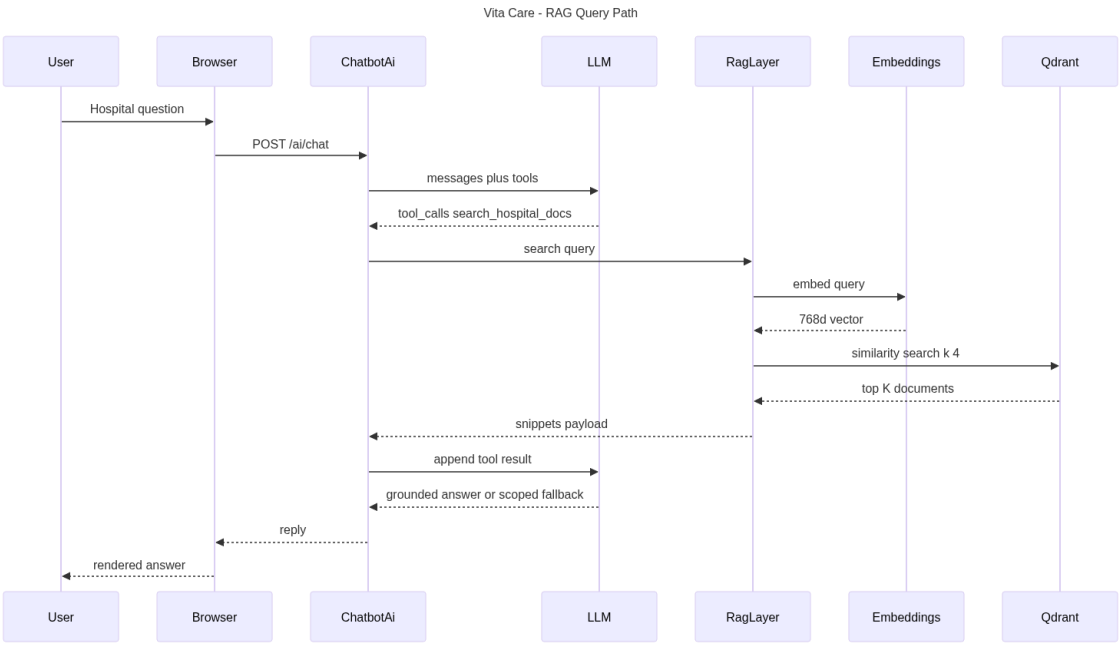


Figure 4 — Hospital FAQ via retrieval (RAG)

```
PSEUDOCODE (sketch):
  snippets ← similarity_search(user_question)
  IF hospital-specific AND snippets empty → refuse to invent policy
  ELSE answer from snippets (or prefixed general knowledge)
```

Full pseudocode and code snippets: Appendix E.2 RAG retrieval.

7.4 Guided booking in the widget

Card steps (department → date → slot → name → confirm) avoid LLM latency and cost. Free text uses the full chat path; when the backend returns structured events, the UI returns users to the appropriate card step.

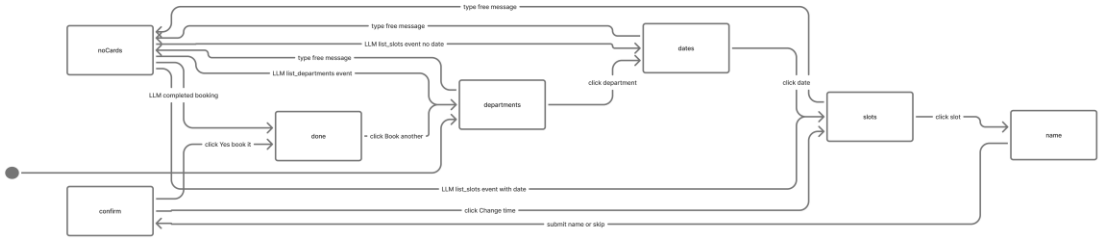


Figure 5 — Guided booking in the chat widget

```
PSEUDOCODE (sketch):
  cards: dept → date → slot → name → POST /api/bookings
  free text: POST /ai/chat → applyLlmEvents(events)
    → reopen cards at departments / dates / slots when tools say so
```

Full pseudocode and code snippets: Appendix E.6 Guided booking in the UI.

7.5 Server orchestration

The chat entry point caps history for provider limits, allows several tool rounds, applies guards, and may emit UI events. Emergency phrases bypass the model with a fixed escalation message.

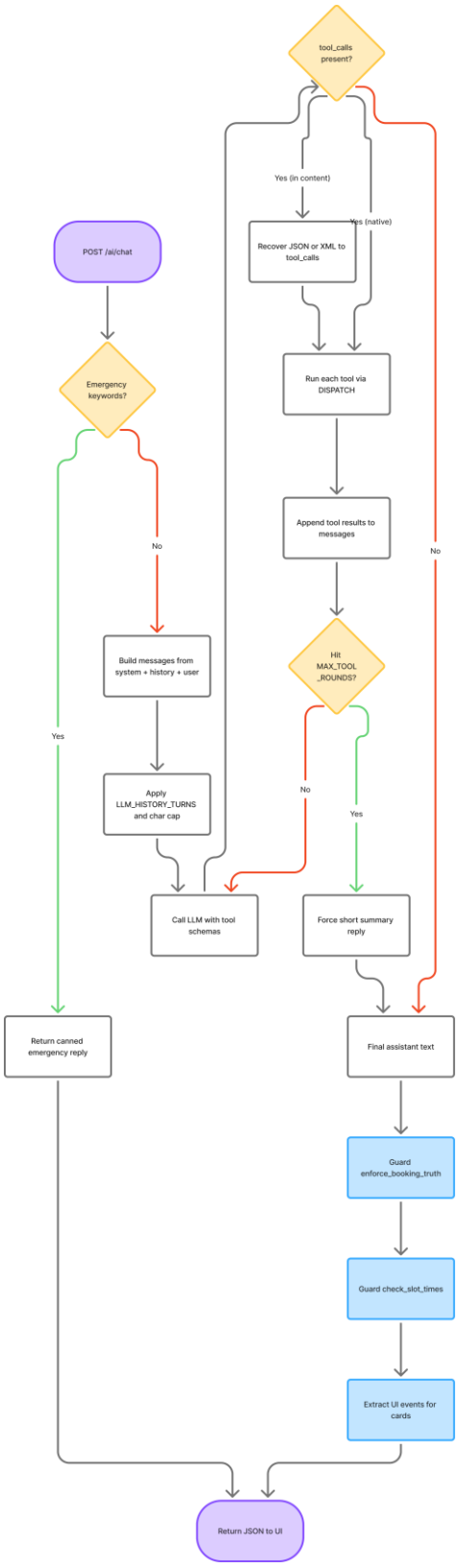


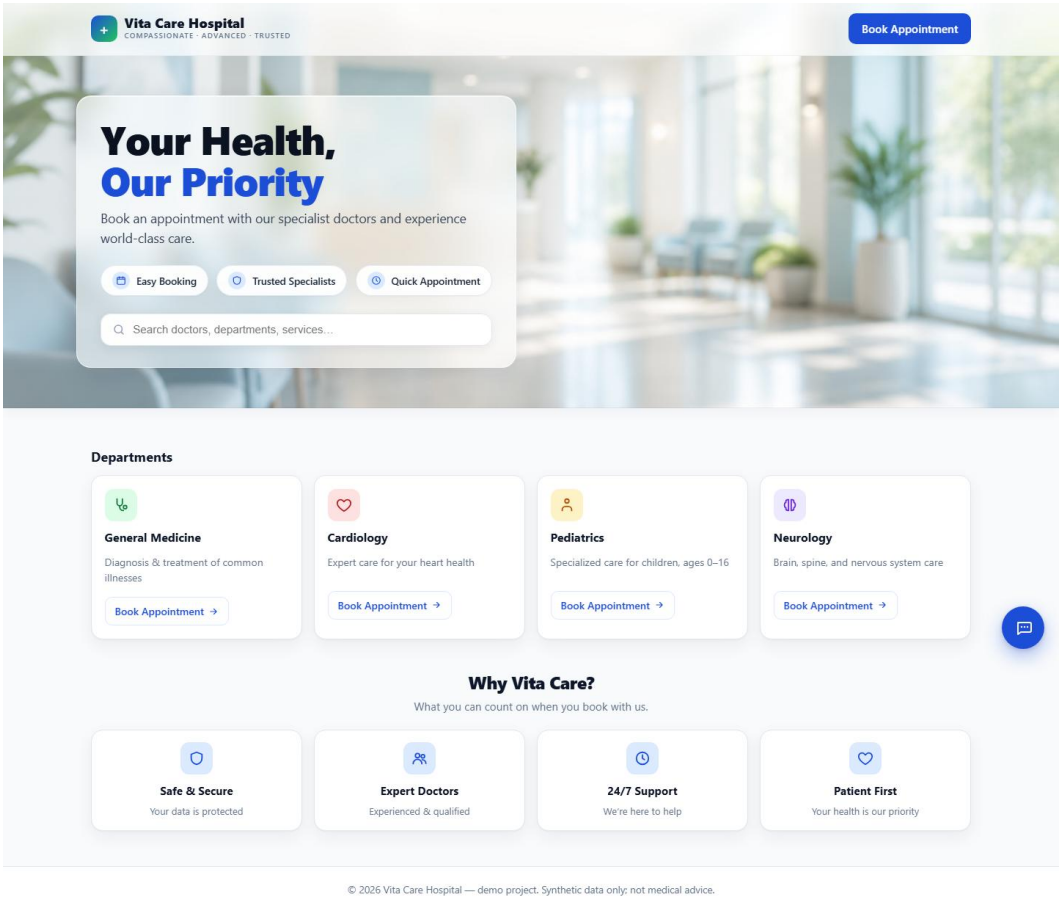
Figure 6 — Chat orchestration on the server

```
PSEUDOCODE (sketch):
  IF emergency_phrase → fixed reply; STOP
  LOOP (≤4): LLM → tools OR final text
  APPLY booking_truth + slot_time guards
  RETURN { reply, events for UI }
```

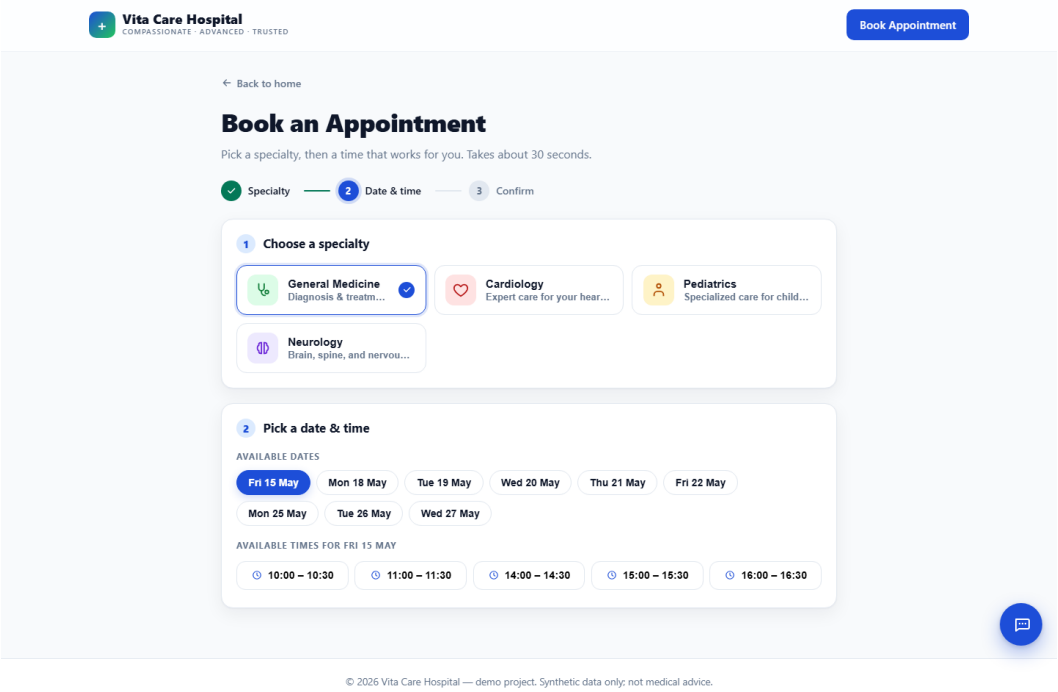
Full pseudocode and code snippets: Appendix E.4 Chat orchestration — run_chat.

Appendix A — UI screenshots

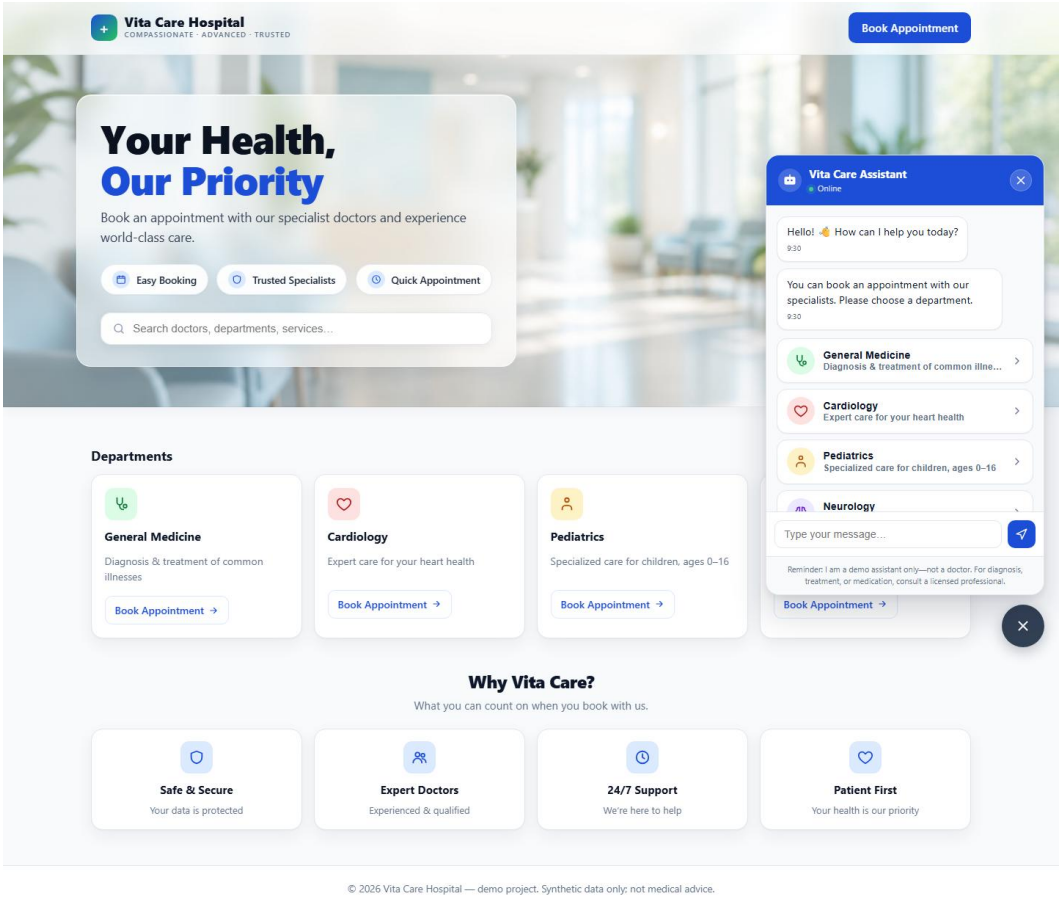
Representative screens from the running demo (repo: docs/screenshots/).



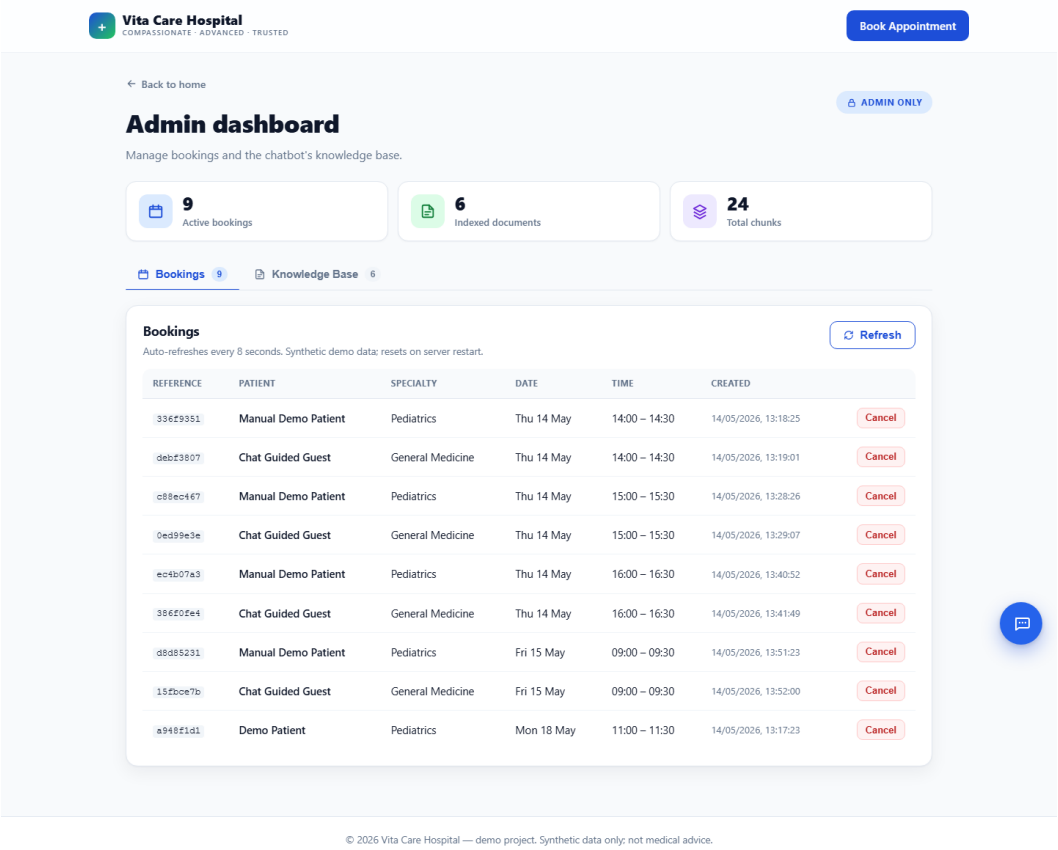
Home — department cards



Manual booking flow



Assistant open



Admin — bookings and knowledge base

Appendix B — HTTP interface reference

Booking API (hospital-api, prefix /api):

- GET /api/departments — list departments.
- GET /api/slots?department_id=… — available dates and slot identifiers.
- POST /api/bookings — create booking (patient_name, department_id, slot_id).
- DELETE /api/bookings/{id} — cancel booking.

Chat API (chatbot-ai, prefix /ai):

- POST /ai/chat — body: message, optional history → reply and optional UI events.
- POST /ai/admin/ingest — multipart upload; optional Bearer ADMIN_TOKEN.
- GET /ai/admin/documents — list ingested sources and chunk counts.

Default local ports: UI 5173, chat 8000, booking API 8001, Qdrant 6333, Ollama 11434.

Appendix C — Component reference

Component	Port (default)	Responsibility
hospital-ui	5173	React SPA; proxy; guided chat cards.
hospital-api	8001	Departments, slots, bookings; SQLite.
chatbot-ai	8000	Chat, tools, RAG, admin ingest.
Qdrant	6333	Collection vitacare_docs.
Ollama	11434	Embeddings; optional local chat.
Groq	HTTPS	Optional hosted chat.
SQLite	file	data/hospital.db under hospital-api.

Appendix E — Implementation reference

Pseudocode and trimmed source excerpts for engineering review. Section 7 summarizes each capability in narrative form with a short logic sketch; this appendix holds the detail.

E.1 Knowledge Ingest

Re-uploading the same logical source name replaces prior vectors before new chunks are written.

```
PSEUDOCODE ingest_file(upload):
  source ← logical filename
  delete_source(source)          // remove old Qdrant points for this source
  docs ← load_pdf_or_txt_or_docx
  chunks ← RecursiveCharacterTextSplitter
  vectorstore.add_documents(chunks)
  RETURN { source, chunks: N, replaced_chunks: M }
```

```
def ingest_file(saved_path, source_name=None):
    src = source_name or saved_path.name
    removed = delete_source(src)
    docs = _load_documents(saved_path, src)
    chunks = _split(docs)
    vs.add_documents(chunks)
    return {"source": src, "chunks": len(chunks), "replaced_chunks": removed}
```

Source: /chatbot-ai/app/ingest_service.py (abbreviated)

E.2 RAG retrieval

```
PSEUDOCODE rag.search(query):
  IF collection missing:
    RETURN { snippets: [], error: "knowledge_base_unavailable" }
  docs ← vectorstore.similarity_search(query, top_k)
  FOR each doc:
    snippets.append({ source, content truncated to max_chars })
  RETURN { query, snippets, count }
```

```
async def search(query: str, k: int | None = None) -> dict:
    if not collection_exists():
        return {"snippets": [], "error": "knowledge_base_unavailable: ..."}
    docs = await asyncio.to_thread(vs.similarity_search, q, top_k)
    return {"query": q, "snippets": snippets, "count": len(snippets)}
```

Source: /chatbot-ai/app/rag.py

E.3 Booking via tools & post-reply guards

Tool dispatch and guards keep the model off the database and correct over-confident replies.

```
PSEUDOCODE run_tool_call(name, args):
  SWITCH name:
    list_departments / list_slots / create_booking / …
      → booking_client (HTTP to hospital-api)
    search_hospital_docs → rag.search(query)

PSEUDOCODE enforce_booking_truth(text, create_results):
  IF text claims booking AND no tool returned { id }:
    REPLACE reply with honest failure message

PSEUDOCODE check_slot_times(text, slot_results):
  IF reply mentions times not in last list_slots result:
    APPEND correction with actual start times
```

```
def is_emergency_risk(user_text: str) -> bool:
    return bool(EMERGENCY_PATTERNS.search(user_text or ""))

def _enforce_booking_truth(text, create_results):
    if not _CLAIMED_BOOKING_RE.search(text):
        return text
    any_ok, errors = _summarise_create_booking_results(create_results)
    if any_ok:
        return text
    return notice
```

Source: /chatbot-ai/app/guardrails.py, chat_service.py, tools.py

E.4 Chat orchestration — run_chat

One user message becomes a reply plus optional UI events for the widget.

```
PSEUDOCODE run_chat(user_message, history):
  IF is_emergency_risk(user_message):
    RETURN fixed_emergency_reply, events=[]
  messages ← system_prompt + truncated(history) + user_message
  FOR up to 4 rounds:
    response ← LLM(messages, tools=booking_tools)
    IF no tool_calls: final_text ← response.content; BREAK
    FOR each tool_call:
      result ← run_tool_call(name, args)
      track create_booking / list_slots; append UI events when relevant
  IF final_text empty: final_text ← LLM(messages, tools=none)
  final_text ← enforce_booking_truth(...); final_text ← check_slot_times(...)
  RETURN { reply, events }
```

```
async def run_chat(user_message, history):
    if is_emergency_risk(user_message):
        return {"reply": EMERGENCY_REPLY, "events": []}
    for _ in range(MAX_TOOL_ROUNDS):
        msg = await ollama_chat(messages, tools=tools)
        if not msg.get("tool_calls"):
            final_text = msg.get("content", "").strip()
            break
        for call in msg["tool_calls"]:
            result_json = await run_tool_call(name, args)
            ...
    final_text = _enforce_booking_truth(final_text, create_booking_results)
    return {"reply": final_text, "events": ui_events}
```

Source: /chatbot-ai/app/chat_service.py (abbreviated)

E.5 Booking transaction — SQLite

Slot availability is enforced in the database with an immediate transaction — two users cannot take the same slot.

```
PSEUDOCODE create_booking(patient_name, department_id, slot_id):
  BEGIN IMMEDIATE
  row ← SELECT slot WHERE id = slot_id
  IF row missing OR wrong department OR NOT available:
    ROLLBACK; RAISE error
  UPDATE slots SET available = 0 WHERE id = slot_id AND available = 1
  IF rowcount ≠ 1: ROLLBACK; RAISE slot_unavailable
  INSERT INTO bookings (id, patient_name, department_id, slot_id, created_at)
  COMMIT
  RETURN { id, department_name, start, end, message }
```

```
with self._connect() as cx:
```

```
    cx.execute("BEGIN IMMEDIATE")
    row = cx.execute("SELECT * FROM slots WHERE id = ?", (slot_id,)).fetchone()
    if not row["available"]:
        cx.execute("ROLLBACK"); raise ValueError("slot_unavailable")
    cur = cx.execute(
        "UPDATE slots SET available = 0 WHERE id = ? AND available = 1 ...")
    cx.execute("INSERT INTO bookings (...) VALUES (...)", ...)
    return {"id": bid, "start": start_iso, ...}
```

Source: /hospital-api/app/sqlite_store.py (abbreviated)

E.6 Guided booking in the UI

Card steps call the booking API directly (no LLM). Free text clears the card flow and uses /ai/chat; tool events from the response can re-open cards at the right step.

```
PSEUDOCODE guided flow state:
  step ∈ { departments, dates, slots, name, confirm, done }
  step = null  → free-chat mode (cards hidden)

on department picked:
  slotData ← GET /api/slots?department_id
  step ← dates

on date picked → step ← slots; on slot picked → step ← name
on name entered → step ← confirm → POST /api/bookings → step ← done

on free-text send:
  step ← null
  { reply, events } ← POST /ai/chat
  applyLlmEvents(events):
    IF last create_booking succeeded → clear flow
    ELIF last list_slots → step ← dates or slots
    ELIF last list_departments → step ← departments
```

```
// flow.step: "departments" | "dates" | "slots" | "name" | "confirm" | null
const applyLlmEvents = (events) => {
  const lastBooking = [...events].reverse().find(
    (e) => e.name === "create_booking" && e.result?.id);
  if (lastBooking) { setFlow({ step: null, ... }); return; }
  const lastSlots = [...events].reverse().find(
    (e) => e.name === "list_slots" && e.result?.slots);
  if (lastSlots) { setFlow({ step: requestedDate ? "slots" : "dates", ... }); }
};
```

Source: /hospital-ui/src/components/Chatbot.jsx (abbreviated)